



UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Departamento Académico de Informática
Escuela Profesional: Ing. Informática y de Sistemas

COMPUTACIÓN GRÁFICA I

Práctica N°10

TRANSFORMACIONES EN EL ESPACIO 3D

Traslación, rotación y escalamiento

Estudiante: Efrain Vitorino Marin

Código: 160337

Profesor: Dr. Hans Harley Ccacyahuilca Bejar

Repositorio: <https://github.com/lovito99/laboGrafica/tree/main/lab010>

Resumen

En esta práctica se implementaron transformaciones tridimensionales usando OpenGL moderno, GLFW, GLEW y GLM. La actividad consiste en rotar un cubo alrededor de la recta que pasa por $A = [0, 0, 0]$ y $B = [7, 7, 7]$, además de rotar un tetraedro alrededor de la recta definida por los puntos $P = [2, 1, 0]$ y $Q = [6, 5, 0]$.

Objetivos

- Implementar transformaciones 3D mediante matrices homogéneas de 4×4 .
- Usar el esquema `model`, `view` y `projection` de OpenGL moderno.
- Derivar y aplicar matrices de rotación alrededor de una línea arbitraria.
- Visualizar gráficamente el objeto original y el objeto transformado.

Librerías usadas

La implementación sigue la plantilla del PDF de la práctica. Se usan GLFW para la ventana, GLEW para cargar extensiones de OpenGL y GLM para el manejo de vectores y matrices.

Listing 1: Librerías principales usadas en la practica.

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
```

Base teórica

Una rotación alrededor de una línea arbitraria se puede construir trasladando la recta al origen, aplicando la rotación respecto a su vector director y retornando luego a la posición inicial.

Si la recta pasa por los puntos A y B , su vector director unitario es:

$$\vec{u} = \frac{B - A}{\|B - A\|}$$

Para una recta que no necesariamente pasa por el origen, la matriz global es:

$$M = T(A) R(\vec{u}, \theta) T(-A)$$

La matriz $R(\vec{u}, \theta)$ se obtiene con la fórmula de Rodrigues. Si $\vec{u} = (x, y, z)$, $c = \cos \theta$, $s = \sin \theta$ y $k = 1 - c$, entonces:

$$R = \begin{pmatrix} c + x^2k & xyk - zs & xzk + ys & 0 \\ yxk + zs & c + y^2k & yzk - xs & 0 \\ zxk - ys & zyk + xs & c + z^2k & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Actividad 1: Cubo rotado

El primer ejercicio solicita rotar un cubo 30° alrededor de la línea que pasa por:

$$A = [0, 0, 0], \quad B = [7, 7, 7]$$

El vector director es:

$$B - A = [7, 7, 7]$$

Normalizando:

$$\vec{u} = \left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}} \right)$$

Como la recta pasa por el origen, la matriz de transformación no requiere traslación adicional:

$$M = R(\vec{u}, 30^\circ)$$

Listing 2: Construcción de la matriz para el cubo.

```
const glm::vec3 a(0.0f, 0.0f, 0.0f);
const glm::vec3 b(7.0f, 7.0f, 7.0f);
const glm::mat4 rot = matRotLinea(a, b, glm::radians(30.0f));
```

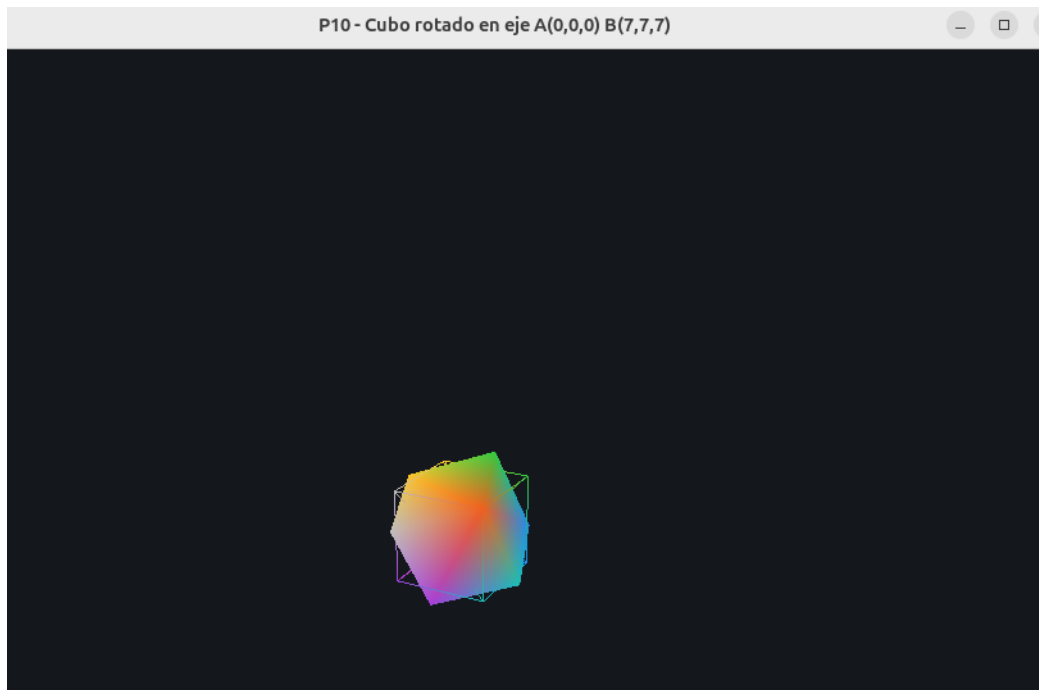


Figura 1: Cubo original y cubo rotado 30° alrededor de la recta $A(0,0,0)-B(7,7,7)$.

Actividad 2: Tetraedro rotado

El segundo ejercicio solicita rotar un tetraedro 45° alrededor de la línea que pasa por:

$$P = [2, 1, 0], \quad Q = [6, 5, 0]$$

El vector director es:

$$Q - P = [4, 4, 0]$$

Normalizando:

$$\vec{u} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0 \right)$$

Como la recta no pasa por el origen, se traslada el punto P al origen, se aplica la rotación y luego se retorna:

$$M = T(2, 1, 0) R(\vec{u}, 45^\circ) T(-2, -1, 0)$$

Listing 3: Construcción de la matriz para el tetraedro.

```
const glm::vec3 p(2.0f, 1.0f, 0.0f);
const glm::vec3 q(6.0f, 5.0f, 0.0f);
const glm::mat4 rot = matRotLinea(p, q, glm::radians(45.0f));
```

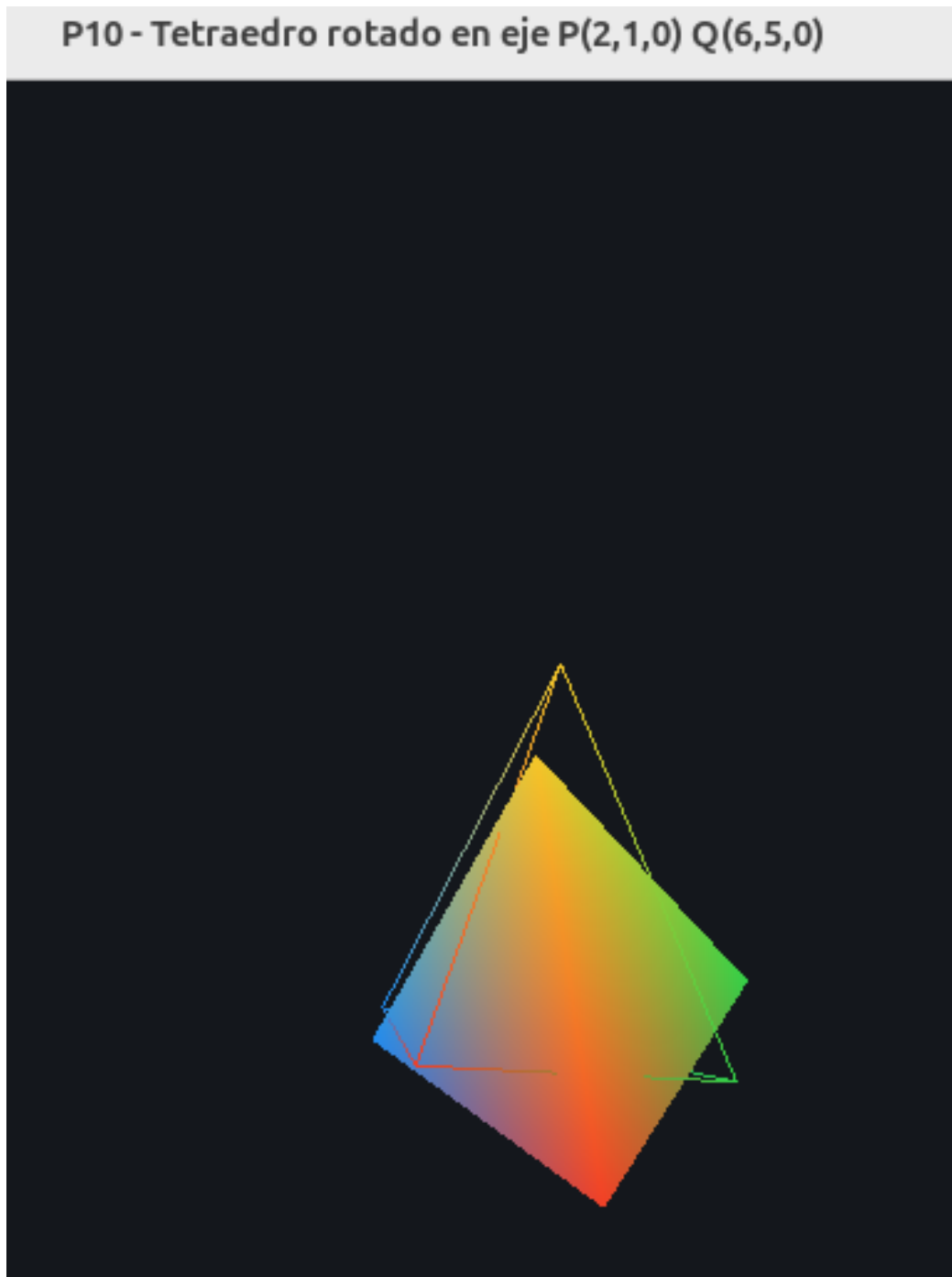


Figura 2: Tetraedro original y tetraedro rotado 45° alrededor de la recta $P(2,1,0)$ – $Q(6,5,0)$.

Implementación

El archivo `p10base.hpp` centraliza los shaders, la configuración de OpenGL y las funciones matemáticas. Las actividades se implementan en `actCubo.cpp` y `actTetra.cpp`.

Vertex Shader y Fragment Shader

El *vertex shader* recibe la posición y el color de cada vértice. La posición final se calcula multiplicando las matrices `projection`, `view` y `model`, como indica la plantilla del PDF.

Listing 4: Vertex shader usado en la practica.

```
const char* const shaderVert = R"glsl(
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;

out vec3 vertexColor;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
{
    gl_Position = projection * view * model * vec4(aPos, 1.0);
    vertexColor = aColor;
}
)glsl";
```

Listing 5: Fragment shader usado para colorear cada cara.

```
const char* const shaderFrag = R"glsl(
#version 330 core
in vec3 vertexColor;
out vec4 FragColor;

void main()
{
    FragColor = vec4(vertexColor, 1.0);
}
)glsl";
```

Configuración de ventana y contexto

La ventana se crea con OpenGL Core Profile 3.3. También se activa `GL_DEPTH_TEST` para que las figuras tridimensionales se dibujen respetando la profundidad.

Listing 6: Creacion de ventana moderna con GLFW y GLEW.

```
inline GLFWwindow* crearVent(const char* titulo)
{
    configGlfw();
```

```
if (!glfwInit()) {
    std::cerr << "Error al iniciar GLFW" << std::endl;
    return nullptr;
}

glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

GLFWwindow* vent = glfwCreateWindow(ancho, alto, titulo, nullptr, nullptr);
if (!vent) {
    glfwTerminate();
    return nullptr;
}

glfwMakeContextCurrent(vent);
glViewport(0, 0, ancho, alto);

glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK) {
    glfwDestroyWindow(vent);
    glfwTerminate();
    return nullptr;
}

glEnable(GL_DEPTH_TEST);
return vent;
}
```

Matriz de traslación y rotación

La traslación se construye manualmente como una matriz homogénea. Esto mantiene coherencia con el desarrollo teórico del PDF.

Listing 7: Matriz de traslación manual.

```
inline glm::mat4 matTras(float tx, float ty, float tz)
{
    return glm::mat4(
        glm::vec4(1.0f, 0.0f, 0.0f, 0.0f),
        glm::vec4(0.0f, 1.0f, 0.0f, 0.0f),
        glm::vec4(0.0f, 0.0f, 1.0f, 0.0f),
        glm::vec4(tx, ty, tz, 1.0f));
}
```

La rotación alrededor de un eje arbitrario se implementa con la fórmula de Rodrigues. Primero se normaliza el eje y luego se colocan los coeficientes en una matriz compatible con GLM.

Listing 8: Rotacion alrededor de un eje arbitrario.

```
inline glm::mat4 matRotEje(glm::vec3 eje, float angRad)
{
    eje = glm::normalize(eje);

    const float x = eje.x;
    const float y = eje.y;
    const float z = eje.z;
    const float c = std::cos(angRad);
    const float s = std::sin(angRad);
    const float omc = 1.0f - c;

    return glm::mat4(
        glm::vec4(c + x * x * omc, y * x * omc + z * s, z * x * omc - y * s, 0.0f),
        glm::vec4(x * y * omc - z * s, c + y * y * omc, z * y * omc + x * s, 0.0f),
        glm::vec4(x * z * omc + y * s, y * z * omc - x * s, c + z * z * omc, 0.0f),
        glm::vec4(0.0f, 0.0f, 0.0f, 1.0f));
}
```

Listing 9: Funcion general para rotar alrededor de una linea.

```
inline glm::mat4 matRotLinea(glm::vec3 a, glm::vec3 b, float angRad)
{
    const glm::vec3 eje = b - a;
    return matTras(a.x, a.y, a.z)
        * matRotEje(eje, angRad)
        * matTras(-a.x, -a.y, -a.z);
}
```

Creación de VAO, VBO y EBO

La geometría se envía a la GPU mediante un VAO, un VBO y un EBO. Cada vértice contiene seis valores: tres para posición y tres para color.

Listing 10: Carga de vertices e indices en buffers de OpenGL.

```
inline Malla crearMalla(const float* verts, GLsizeiptr tamVerts,
                        const unsigned int* inds, GLsizeiptr tamInds,
                        GLsizei cantInds)
```

```
{
    Malla malla;
    malla.cant = cantInds;

    glGenVertexArrays(1, &malla.vao);
    glGenBuffers(1, &malla.vbo);
    glGenBuffers(1, &malla.ebo);

    glBindVertexArray(malla.vao);
    glBindBuffer(GL_ARRAY_BUFFER, malla.vbo);
    glBufferData(GL_ARRAY_BUFFER, tamVerts, verts, GL_STATIC_DRAW);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, malla.ebo);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, tamInds, inds, GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
    ;
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float),
                          (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);

    glBindVertexArray(0);
    return malla;
}
```

Vista y proyección

La matriz `view` se obtiene con `glm::lookAt`. La matriz `projection` se construye mediante una perspectiva manual, siguiendo la idea del programa base del PDF.

Listing 11: Envío de matrices `view` y `projection` al shader.

```
inline void enviarCam(GLuint prog)
{
    const glm::mat4 view = glm::lookAt(
        glm::vec3(10.0f, 9.0f, 18.0f),
        glm::vec3(1.5f, 1.5f, 1.5f),
        glm::vec3(0.0f, 1.0f, 0.0f));

    const float fov = glm::radians(45.0f);
    const glm::mat4 projection = matPersp(
        fov,
        static_cast<float>(ancho) / static_cast<float>(alto),
        0.1f,
        100.0f);
}
```

```

glUniformMatrix4fv(glGetUniformLocation(prog, "view"),
                  1, GL_FALSE, glm::value_ptr(view));
glUniformMatrix4fv(glGetUniformLocation(prog, "projection"),
                  1, GL_FALSE, glm::value_ptr(projection));
}

```

Implementación del cubo

El cubo se define con ocho vértices. Los índices forman las seis caras usando dos triángulos por cara.

Listing 12: Vertices principales del cubo con colores.

```

const float verts[] = {
    -1.0f, -1.0f, -1.0f, 0.90f, 0.15f, 0.12f,
    1.0f, -1.0f, -1.0f, 0.12f, 0.55f, 0.95f,
    1.0f, 1.0f, -1.0f, 0.20f, 0.80f, 0.25f,
    -1.0f, 1.0f, -1.0f, 0.98f, 0.78f, 0.18f,
    -1.0f, -1.0f, 1.0f, 0.65f, 0.25f, 0.86f,
    1.0f, -1.0f, 1.0f, 0.10f, 0.75f, 0.72f,
    1.0f, 1.0f, 1.0f, 0.94f, 0.38f, 0.12f,
    -1.0f, 1.0f, 1.0f, 0.75f, 0.75f, 0.75f
};

```

Listing 13: Indices del cubo: seis caras con dos triángulos por cara.

```

const unsigned int inds[] = {
    0, 1, 2, 0, 2, 3,
    4, 5, 6, 4, 6, 7,
    0, 4, 7, 0, 7, 3,
    1, 5, 6, 1, 6, 2,
    3, 2, 6, 3, 6, 7,
    0, 1, 5, 0, 5, 4
};

```

Implementación del tetraedro

El tetraedro se modela con cuatro vértices y cuatro caras triangulares. La transformación se calcula a partir de los puntos P y Q .

Listing 14: Vertices e indices del tetraedro.

```

const float verts[] = {
    2.0f, 1.0f, 3.0f, 0.95f, 0.25f, 0.15f,
    0.0f, 0.0f, 0.0f, 0.12f, 0.55f, 0.95f,

```

```
    4.0f, 0.0f, 0.0f, 0.20f, 0.82f, 0.28f,  
    2.0f, 4.0f, 0.0f, 0.96f, 0.78f, 0.16f  
};  
  
const unsigned int inds[] = {  
    0, 1, 2,  
    0, 2, 3,  
    0, 3, 1,  
    1, 3, 2  
};
```

En el bucle de renderizado se dibuja primero la figura original en modo alambre y luego la figura transformada con caras coloreadas:

Listing 15: Dibujo del objeto original y transformado.

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);  
dibujar(cubo, modelLoc, glm::mat4(1.0f), GL_TRIANGLES);  
  
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);  
dibujar(cubo, modelLoc, rot, GL_TRIANGLES);
```

Conclusiones

- Se implementó correctamente la rotación 3D alrededor de líneas arbitrarias.
- La actividad del cubo se resolvió usando el eje unitario $(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})$.
- La actividad del tetraedro requirió una composición con traslación porque la recta no pasa por el origen.
- El uso de GLM facilita la transferencia de matrices a los shaders mediante `glm::value_ptr`.